Week2 : Objectives

Upon completion of this unit, students will be able to understand and explain the following sub topics :

- Stacks and Stack Implementations
- push(): Adds an element to the top of the stack.
- pop(): Removes and returns the top element from the stack.
- peek(): Returns the top element without removing it.
- isEmpty(): Checks if the stack contains any elements.
- search(): Determines how far an item is from the top of the stack.

Stacks

- Add item on top of stack
- Remove item that is topmost
  - Last In, First Out … LIFO

- FIGURE 5-1 Some familiar stacks

© 2019 Pearson Education, Inc.

Specifications of the ADT Stack

- Data
  - A collection of objects in reverse chronological order and having the same data type

push(newEntry)	**+push(newEntry: T): void**	Task: Adds a new entry to the top of the stack. Input: newEntry is the new entry. Output: None.
pop()	**+pop(): T**	Task: Removes and returns the stack's top entry. Input: None. Output: Returns the stack's top entry. Throws an exception if the stack is empty before the operation.
peek()	**+peek(): T**	Task: Retrieves the stack's top entry without changing the stack in any way. Input: None. Output: Returns the stack's top entry. Throws an exception if the stack is empty.
isEmpty()	**+isEmpty(): boolean**	Task: Detects whether the stack is empty. Input: None. Output: Returns true if the stack is empty.
clear()	**+clear(): void**	Task: Removes all entries from the stack. Input: None. Output: None.

Design Decision

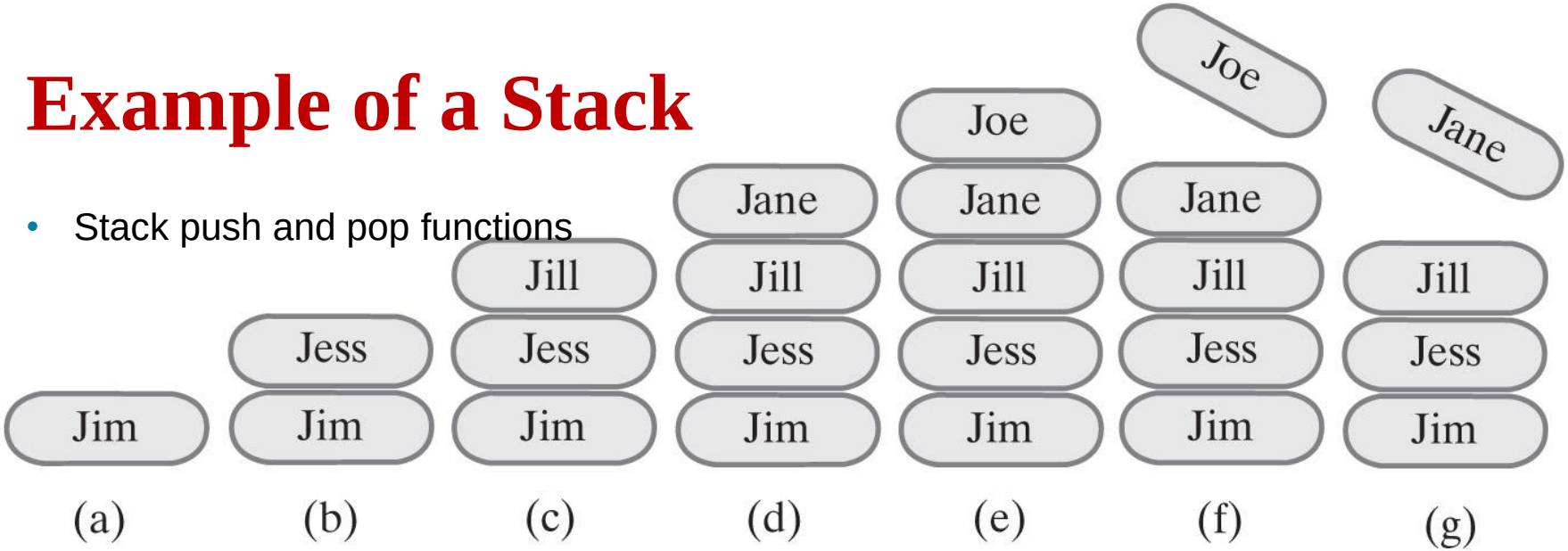
- When stack is empty
  - What to do with **`pop`** and **`peek`**?
- Possible actions
  - Assume that the ADT is not empty;
  - Return null.
  - Throw an exception (which type?).

Interface for the ADT Stack

```
/** An interface for the ADT stack. */
public interface StackInterface<T>
{
   /** Adds a new entry to the top of this stack.
       @param newEntry  An object to be added to the stack. */
   public void push(T newEntry);

   /** Removes and returns this stack's top entry.
       @return  The object at the top of the stack.
       @throws  EmptyStackException if the stack is empty before the operation. */
   public T pop();

   /** Retrieves this stack's top entry.
       @return  The object at the top of the stack.
       @throws  EmptyStackException if the stack is empty. */
   public T peek();

   /** Detects whether this stack is empty.
       @return  True if the stack is empty. */
   public boolean isEmpty();

   /** Removes all entries from this stack. */
   public void clear();
} // end StackInterface
```

Example of a Stack

- Stack push and pop functions



© 2019 Pearson Education, Inc.

```
      StackInterface<String> stringStack = new OurStack<>();
(a)   stringStack.push("Jim");
(b)   stringStack.push("Jess");
(c)   stringStack.push("Jill");
(d)   stringStack.push("Jane");
(e)   stringStack.push("Joe");
(f)   stringStack.pop();
(g)   stringStack.pop();
```

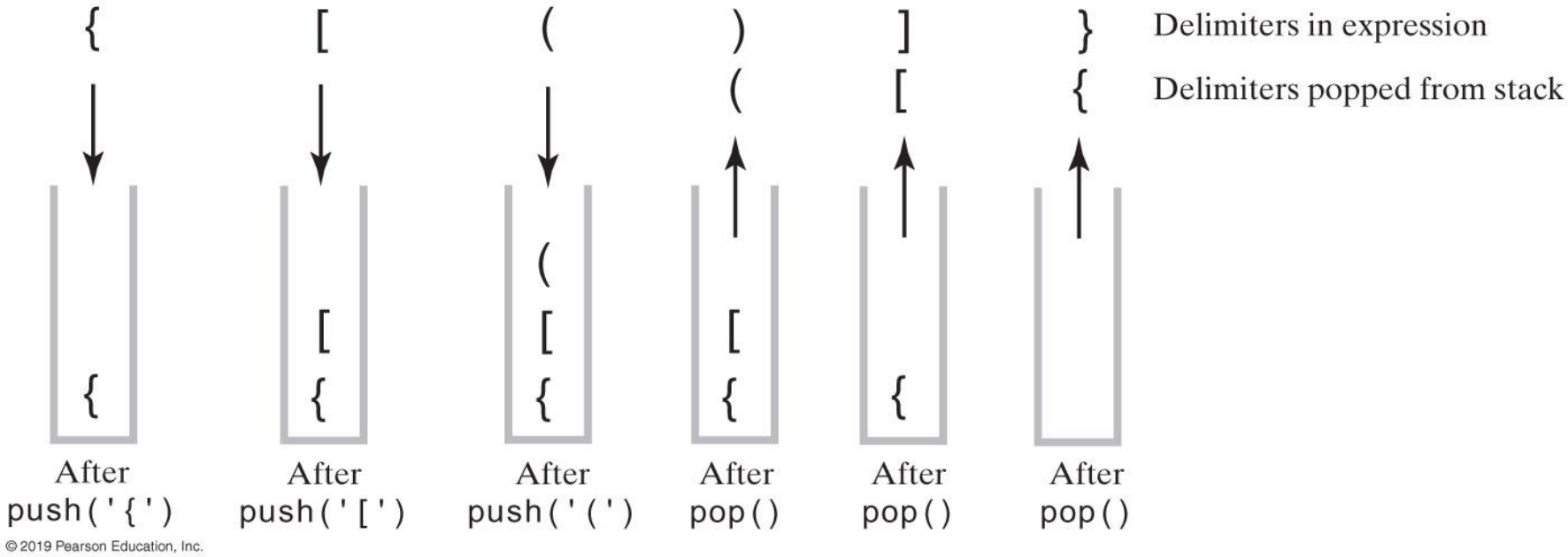
Security Note

- Design guidelines
  - Use preconditions and postconditions to document assumptions.
  - Do not trust client to use public methods correctly.
  - Avoid ambiguous return values.
  - Prefer throwing exceptions instead of returning values to signal problem.

Processing Algebraic Expressions

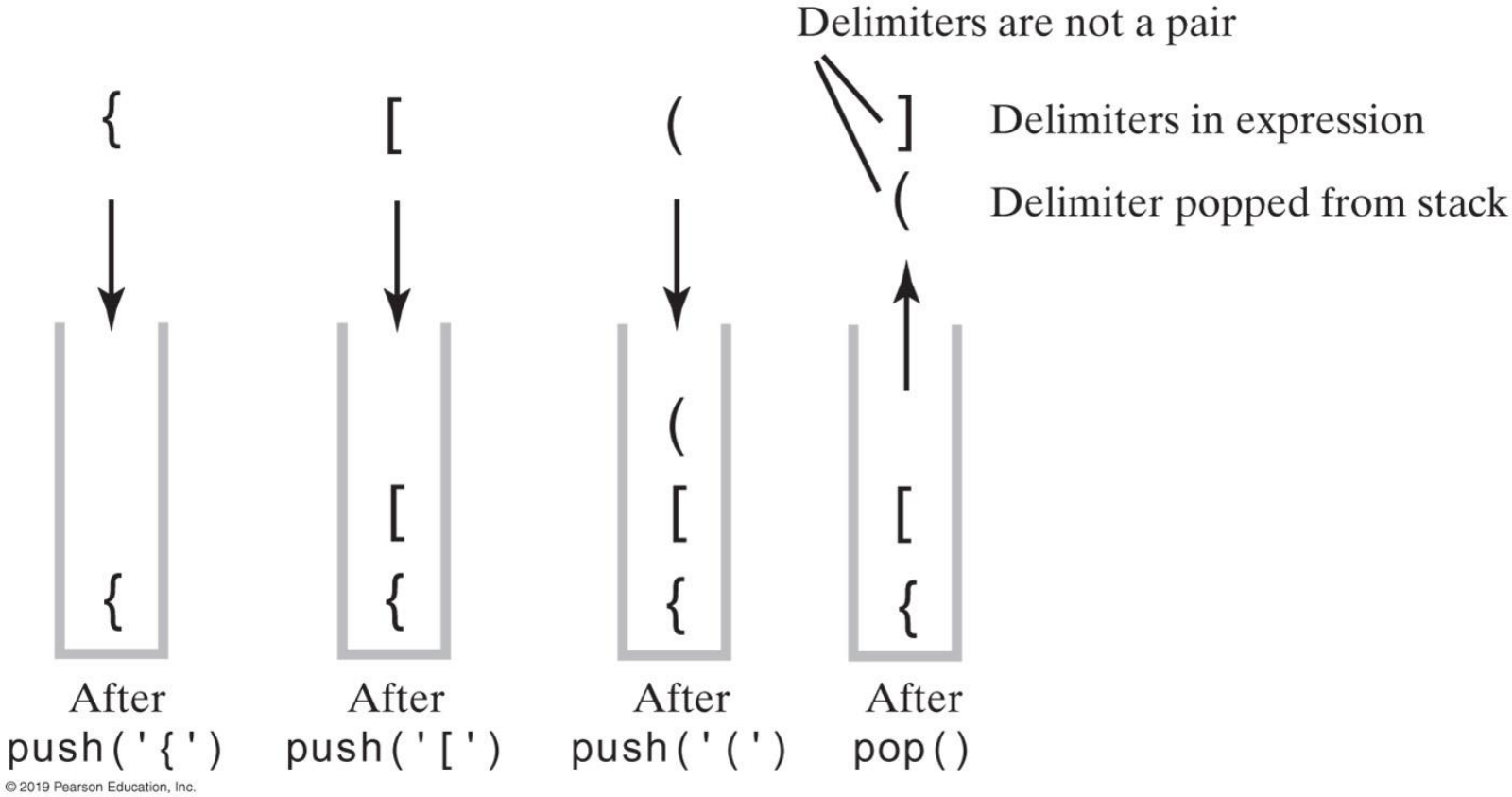
- Infix:
  - each binary operator appears between its operands
    `a + b`
- Prefix:
  - each binary operator appears before its operands
    `+ a b`
- Postfix:
  - each binary operator appears after its operands
    `a b +`
- Balanced expressions: delimiters paired correctly

Processing Algebraic Expressions



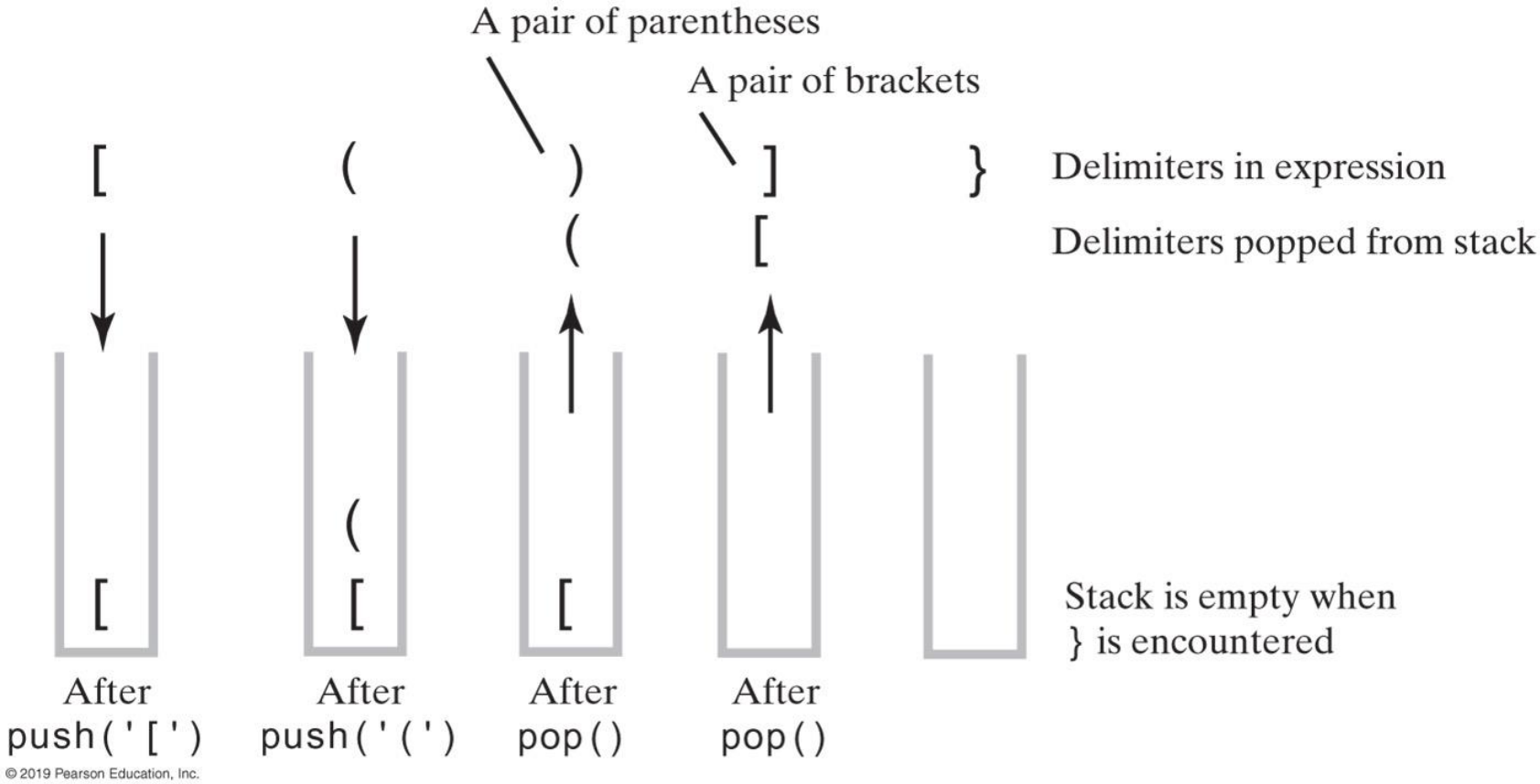
- FIGURE 5-3 The contents of a stack during the scan of an expression that contains the balanced delimiters{ [( )] }

Processing Algebraic Expressions



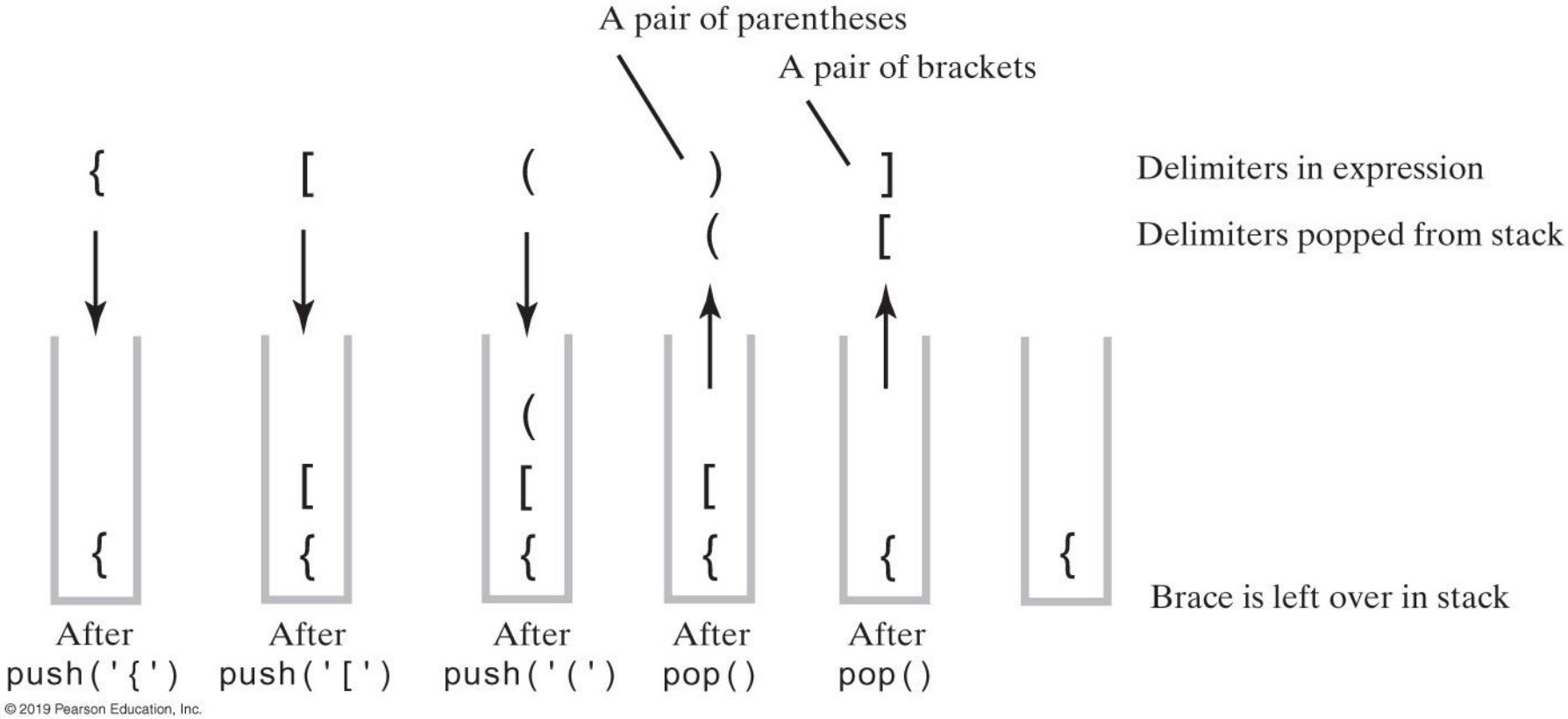
- FIGURE 5-4 The contents of a stack during the scan of an expression that contains the unbalanced delimiters { [( ]) }

Processing Algebraic Expressions



- FIGURE 5-5 The contents of a stack during the scan of an expression that contains the unbalanced delimiters [( )] }

Processing Algebraic Expressions



- FIGURE 5-6 The contents of a stack during the scan of an expression that contains the unbalanced delimiters { [( )]

Processing Algebraic Expressions

```
Algorithm checkBalance(expression)
//Returns true if the parentheses, brackets, and braces in an expression are paired
correctly.

isBalanced = true // The absence of delimiters is balanced
while ((isBalanced == true) and not at end of expression)
{
    nextCharacter = next character in expression
    switch (nextCharacter)
    {
        case '(': case '[': case '{':

                    Push
                nextCharacter    onto
                stack

                    break
        case ')': case ']': case '}':
            if (stack is empty)
                isBalanced = false
            else
            {
                openDelimiter = top entry of stack
                Pop stack
                isBalanced = true or false according to whether openDelimiter and
                             nextCharacter are a pair of delimiters
            }
            break
    }
}
if (stack is not empty)
    isBalanced = false
return isBalanced
```

Implementation of Algorithm (Part 1)

```
/** A class that checks whether the parentheses, brackets, and braces
   in a string occur in left/right pairs. */
public class BalanceChecker
{

   // Returns true if the given characters, open and close, form a pair
   // of parentheses, brackets, or braces.
   private static boolean isPaired(char open, char close)
   {
      return (open == '(' && close == ')') ||
             (open == '[' && close == ']') ||
             (open == '{' && close == '}');
   } // end isPaired
  /** Decides whether the parentheses, brackets, and braces
      in a string occur in left/right pairs.
      @param expression  A string to be checked.
      @return  True if the delimiters are paired correctly. */
   public static boolean checkBalance(String expression)
   {
                                                    [ SEE NEXT SLIDE FOR IMPLEMENTATION]
   } // end checkBalance

} // end BalanceChecker
```

Implementation of Algorithm (Part 2)

```
  StackInterface<Character> openDelimiterStack = new LinkedStack<>();
  int characterCount = expression.length();
  boolean isBalanced = true;
  int index = 0;
  char nextCharacter = ' ';

  while (isBalanced && (index < characterCount)) {
    nextCharacter = expression.charAt(index);
    switch (nextCharacter) {
      case '(': case '[': case '{':
        openDelimiterStack.push(nextCharacter);
        break;
      case ')': case ']': case '}':
        if (openDelimiterStack.isEmpty())
          isBalanced = false;
        else {
          char openDelimiter = openDelimiterStack.pop();
          isBalanced = isPaired(openDelimiter, nextCharacter);
        } // end if
        break;
      default: break; // Ignore unexpected characters
    } // end switch
    index++;
  } // end while

  if (!openDelimiterStack.isEmpty())
    isBalanced = false;

  return isBalanced;
} // end checkBalance
```

Converting Infix to Postfix

`a + b * c`

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
$+$	a	$+$
b	$a\ b$	$+$
$*$	$a\ b$	$+\ *$
c	$a\ b\ c$	$+\ *$
	$a\ b\ c\ *$	$+$
	$a\ b\ c\ *\ +$	

ng

`a ^ b ^ c`

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
^	a	^
b	$a\ b$	^
^	$a\ b$	^ ^
c	$a\ b\ c$	^ ^
	$a\ b\ c$ ^	^
	$a\ b\ c$ ^ ^	

`a - b + c`

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
$-$	a	$-$
b	$a\ b$	$-$
$+$	$a\ b\ -$	
	$a\ b\ -$	$+$
c	$a\ b\ -\ c$	$+$
	$a\ b\ -\ c\ +$	

Converting Infix to Postfix

- FIGURE 5-9 Steps in converting an infix expression to postfix form

```
a / b * (c + (d - e))
```

Next Character from Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
/	a	/
b	a b	/
*	a b /	
	a b /	*
(	a b /	* (
c	a b / c	* (
+	a b / c	* (+
(	a b / c	* (+ (
d	a b / c d	* (+ (
−	a b / c d	* (+ (−
e	a b / c d e	* (+ (−
)	a b / c d e −	* (+ (
	a b / c d e −	* (+
)	a b / c d e − +	* (
	a b / c d e − +	*
	a b / c d e − + *	

Infix-to-postfix Conversion

- To convert an infix expression to postfix form, you take the following actions, according to the symbols you encounter, as you process the infix expression from left to right:

Operand	Append each operand to the end of the output expression.
Operator ^	Push ^ onto the stack.
Operator +, -, *, or /	Pop operators from the stack, appending them to the output expression, until either the stack is empty or its top entry has a lower precedence than the newly encountered operator. Then push the new operator onto the stack.
Open parenthesis	Push (onto the stack.
Close parenthesis	Pop operators from the stack and append them to the output expression until an open parenthesis is popped. Discard both parentheses.

Infix-to-postfix Algorithm (Part 1)

```
Algorithm convertToPostfix(infix)
// Converts an infix expression to an equivalent postfix expression.
operatorStack = a new empty stack
postfix = a new empty string
while (infix has characters left to parse)
{
   nextCharacter = next nonblank character of infix
   switch (nextCharacter)
   {
      case variable:
         Append nextCharacter to postfix
         break
      case '^' :
         operatorStack.push(nextCharacter)
         break
      case '+' : case '-' : case '*' : case '/' :
         while (!operatorStack.isEmpty() and
             precedence of nextCharacter <= precedence of operatorStack.peek())
         {
            Append operatorStack.peek() to postfix
            operatorStack.pop()
         }
        operatorStack.push(nextCharacter)
        break
```

Infix-to-postfix Algorithm (Part 2)

- Algorithm for converting infix to postfix expressions.

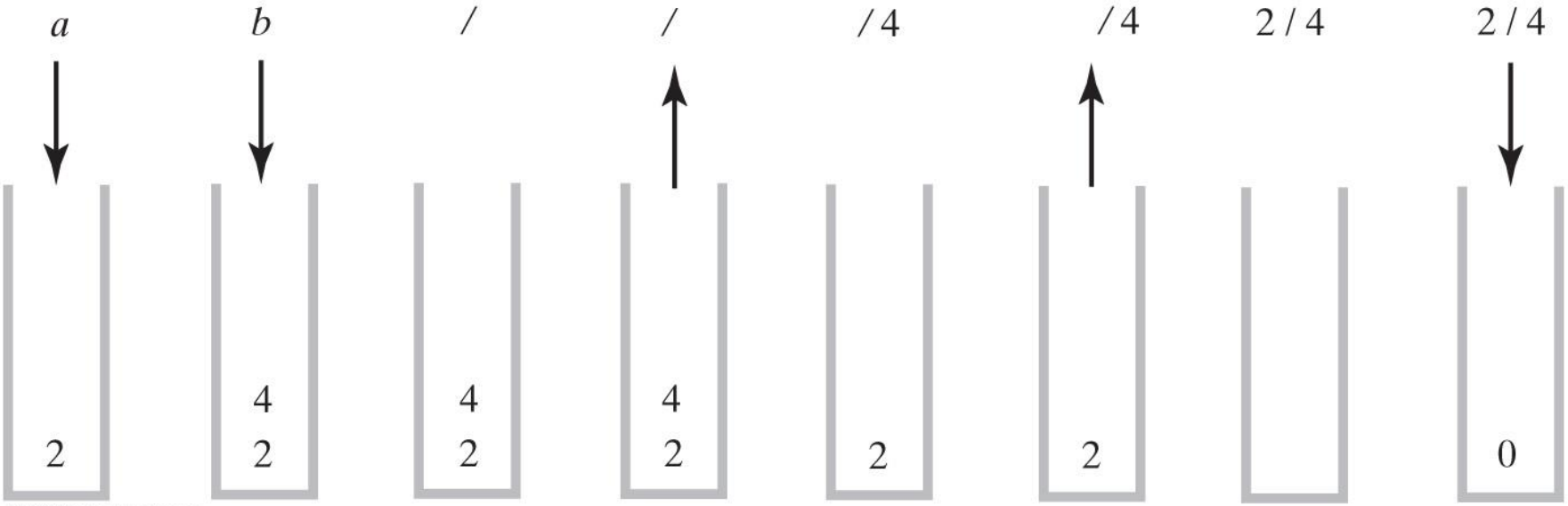
```
            case '( ' :
                        operatorStack.push(nextCharacter)
            break
        case ')' : // Stack is not empty if infix expression is valid
            topOperator = operatorStack.pop()
            while (topOperator != '(')
            {
                Append topOperator to postfix
                topOperator = operatorStack.pop()
            }
            break
        default:
            break // Ignore unexpected characters
    }
}
while (!operatorStack.isEmpty())
{
    topOperator = operatorStack.pop()
    Append topOperator to postfix
}
return postfix
```

Evaluating Postfix Expressions

- Algorithm for evaluating postfix expressions.

```
Algorithm evaluatePostfix(postfix)
// Evaluates a postfix expression.
valueStack = a new empty stack
while (postfix has characters left to parse)
{
   nextCharacter = next nonblank character of postfix
   switch (nextCharacter)
   {
      case variable:
         valueStack.push(value of the variable nextCharacter)
         break
      case '+' : case '-' : case '*' : case '/' : case '^' :
         operandTwo = valueStack.pop()
         operandOne = valueStack.pop()
         result = the result of the operation in nextCharacter and
                                          its operands operandOne and operandTwo
         valueStack.push(result)
         break
      default: break // Ignore unexpected characters
   }
}
return valueStack.peek()
```

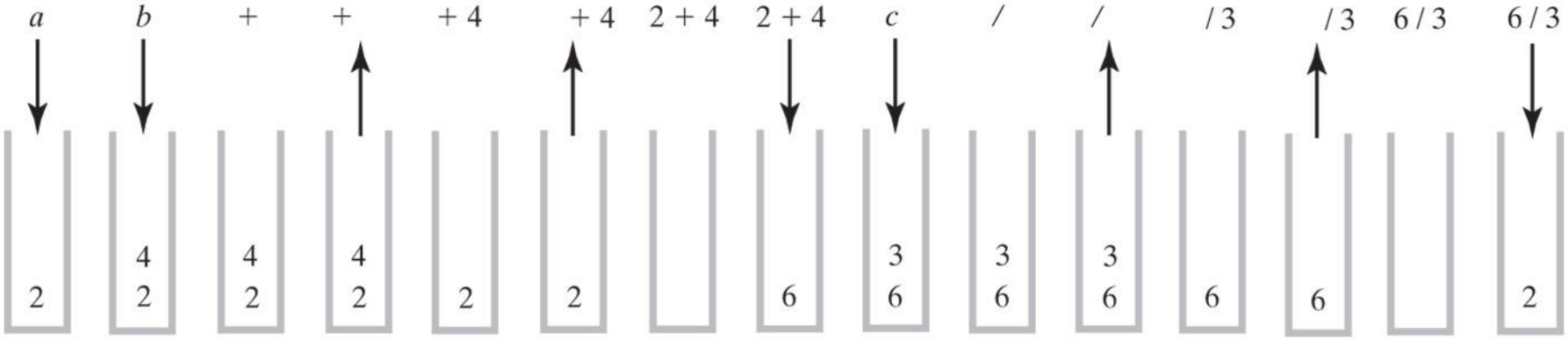
Evaluating Infix Expressions



© 2019 Pearson Education, Inc.

- FIGURE 5-10 The stack during the evaluation of the postfix expression `a b /` when `a` is 2 and `b` is `4`

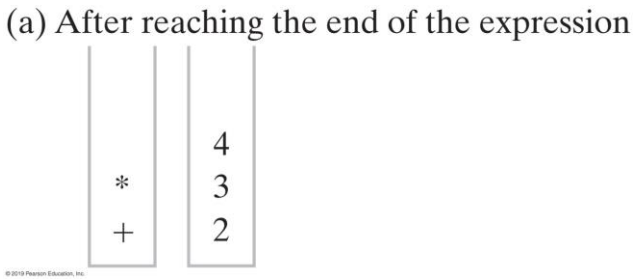
Evaluating Infix Expressions



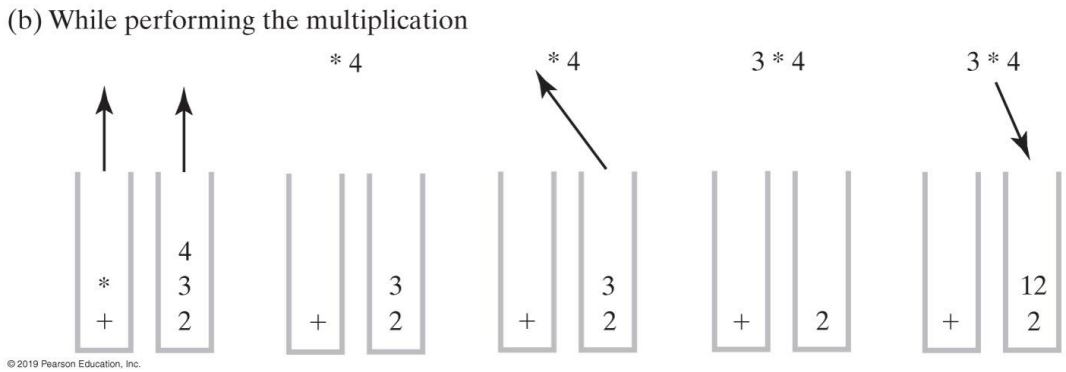
© 2019 Pearson Education, Inc.

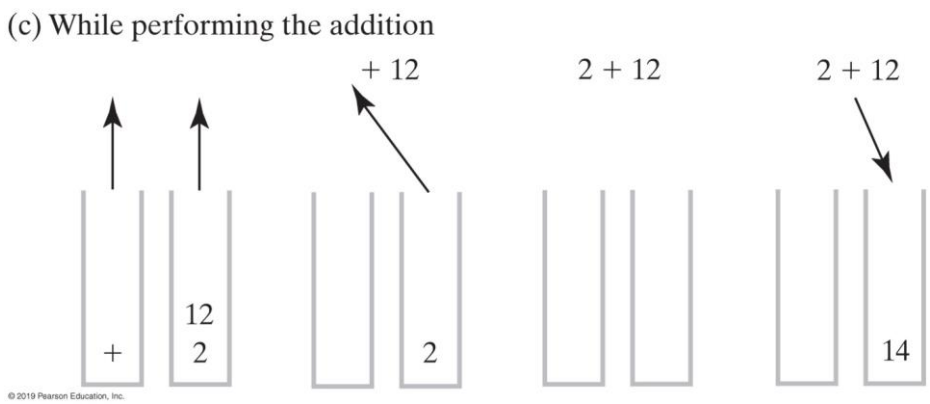
- FIGURE 5-11 The stack during the evaluation of the postfix expression `a b + c /` when `a` is `2`, `b` is `4`, and `c` is `3`

Evaluating Infix Expressions



- FIGURE 5-12
- Two stacks during the evaluation of
- `a + b * c` when `a` is 2, `b` is `3`, and `c` is `4`





Evaluating Infix Expressions (Part 1)

```
Algorithm evaluateInfix(infix)    // Evaluates an infix expression.
operatorStack = a new empty stack
valueStack = a new empty stack
while (infix has characters left to process)
{
   nextCharacter = next nonblank character of infix
   switch (nextCharacter)
   {
      case variable:
         valueStack.push(value of the variable nextCharacter)
         break
      case '^' :
         operatorStack.push(nextCharacter)
         break
      case '+' : case '-' : case '*' : case '/' :
         while (!operatorStack.isEmpty() and
               precedence of nextCharacter <= precedence of operato
         {
            // Execute operator at top of operatorStack
            topOperator = operatorStack.pop()
            operandTwo = valueStack.pop()
            operandOne = valueStack.pop()
```

Evaluating Infix Expressions (Part 2)

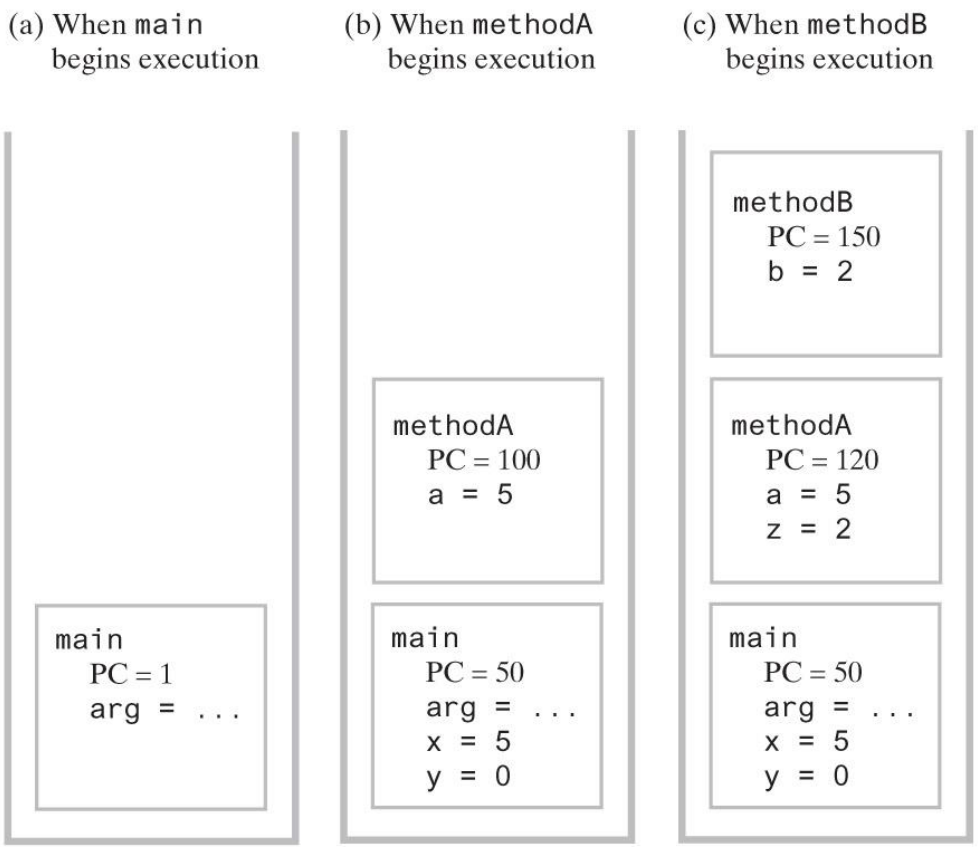
```
        case '(' :
            operatorStack.push(nextCharacter)
            break
        case ')' : // Stack is not empty if infix expression is valid
            topOperator = operatorStack.pop()
            while (topOperator != '(')
            {
                operandTwo = valueStack.pop()
                operandOne = valueStack.pop()
                result = the result of the operation in
                        topOperator and its operands operandOne and op
                valueStack.push(result)
                topOperator = operatorStack.pop()
            }
            break
        default: break // Ignore unexpected characters
    }
}
while (!operatorStack.isEmpty())
{
    topOperator = operatorStack.pop()
    operandTwo = valueStack.pop()
    operandOne = valueStack.pop()
    result = the result of the operation in
                topOperator and its operands operandOne and operandT
```

The Application Program Stack

- F
e

```
1     public static
      void main(string[] arg)
      {
         . . .
         int x = 5;
50       int y = methodA(x);
         . . .
      } // end main

100   public static
      int methodA(int a)
      {
         . . .
         int z = 2;
120       methodB(z);
         . . .
         return z;
      } // end methodA

150   public static
      void methodB(int b)
      {
         . . .
      } // end methodB
```

Program



Program stack at three points in time (PC is the program counter)

© 2019 Pearson Education, Inc.

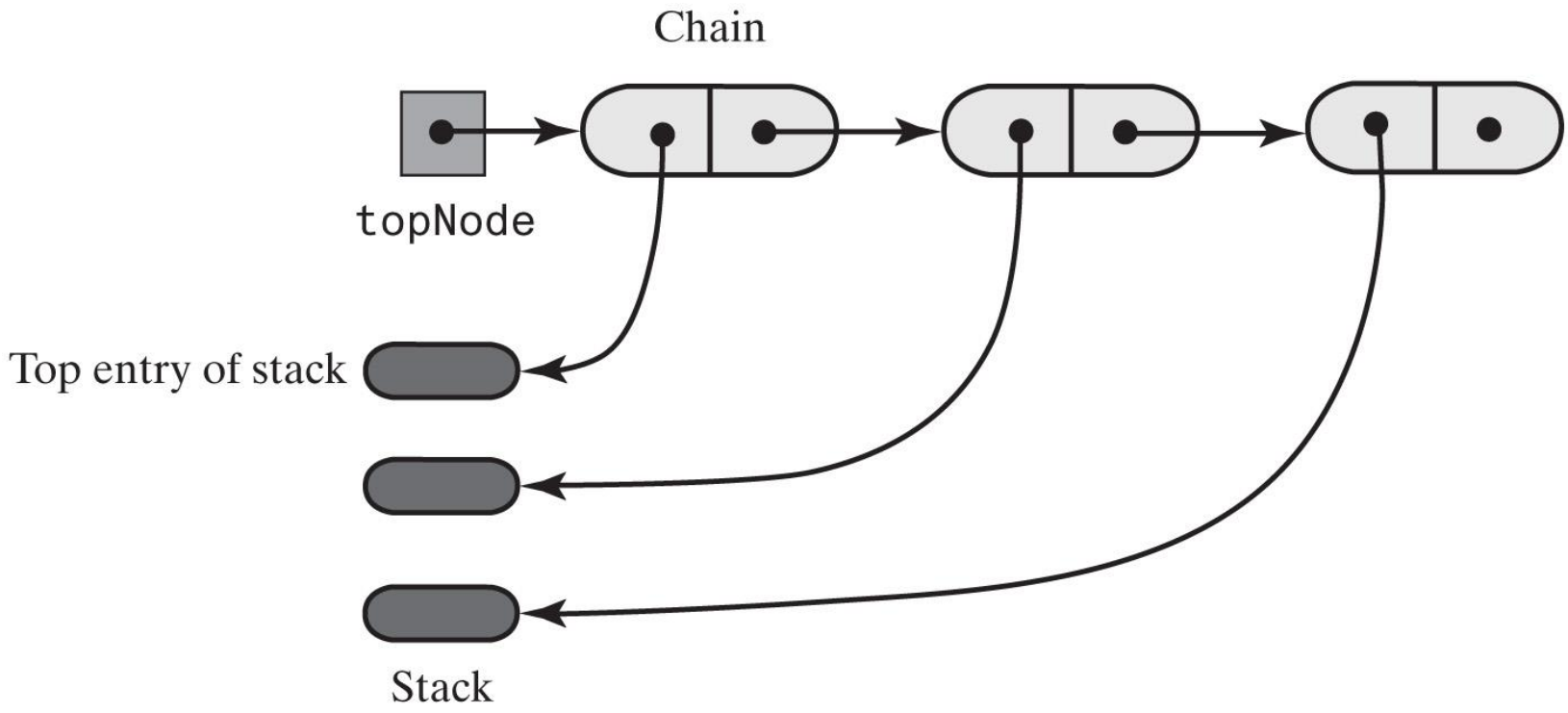
Java Class Library: The Class `Stack`

- Found in `java.util`
- Methods
  - A constructor – creates an empty stack
  - `public T push(T item);`
  - `public T pop();`
  - `public T peek();`
  - `public boolean empty();`

Linked Implementation

- Each operation involves top of stack
  - `push`
  - `pop`
  - `peek`
- Head of linked list easiest, fastest to access
  - Let this be the top of the stack

Linked Implementation



© 2019 Pearson Education, Inc.

Linked Implementation of a Stack

```
/** A class of stacks whose entries are stored in a chain of nodes. */
public final class LinkedStack<T> implements StackInterface<T>
{
   private Node topNode; // References the first node in the chain

   public LinkedStack()
   {
      topNode = null;
   } // end default constructor

//  < Implementations of the stack operations go here. >
//  . . .

        private class Node
        {
      private T    data; // Entry in stack
      private Node next; // Link to next node

                                          //  < Implementations of the node operations go here. >

        } // end Node
} // end LinkedStack
```

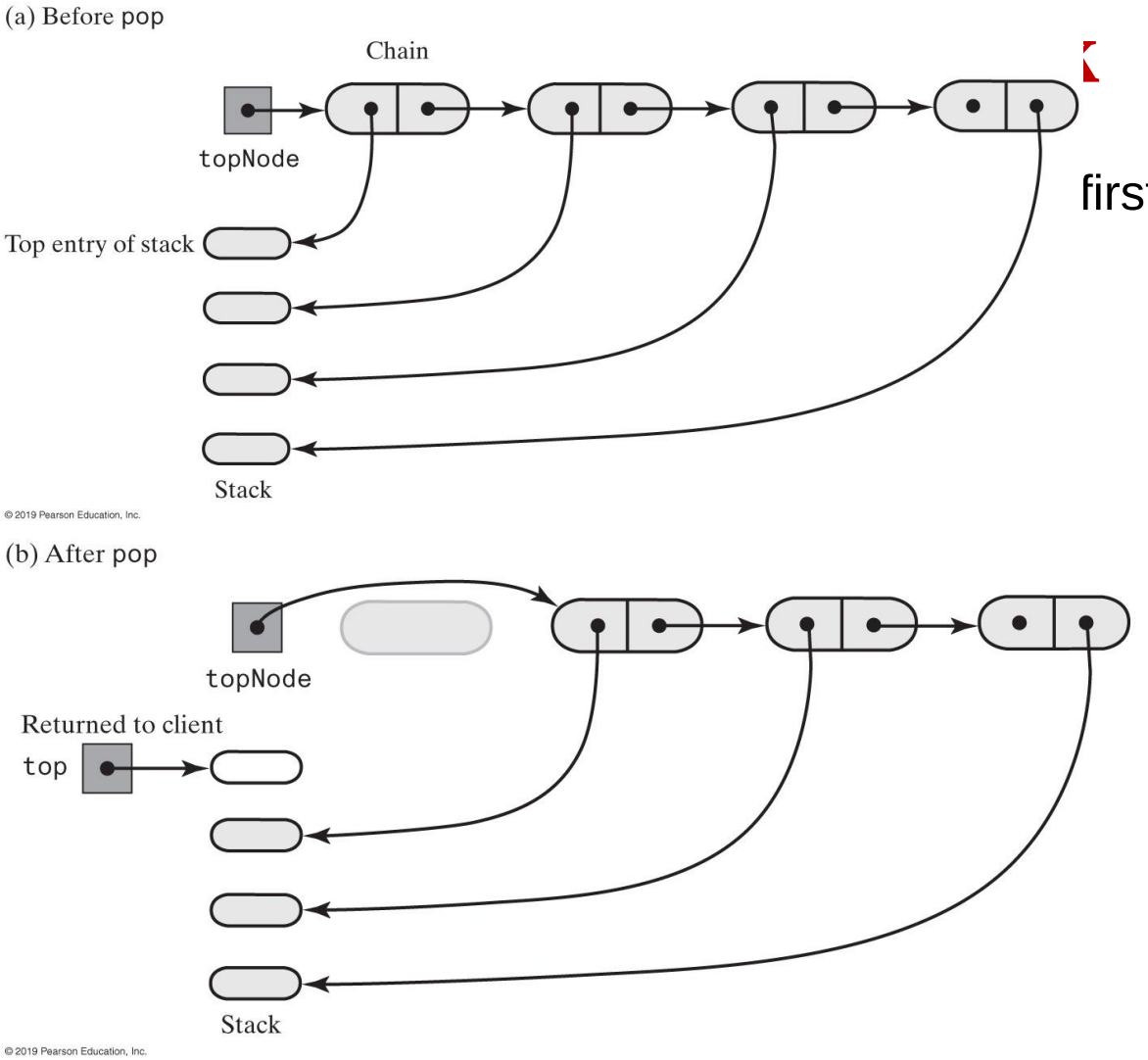
Linked Implementation of a Stack

- Definition of `push`

```
public void push(T newEntry)
{
   Node newNode = new Node(newEntry, topNode);
   topNode = newNode;
} // end push
```

Linke

- FIGUR first node in the chain



Linked Implementation

```
public T peek()
{
   if (isEmpty())
      throw new EmptyStackException();
   else
      return topNode.getData();
} // end peek


public T pop()
{
   T top = peek();  // Might throw EmptyStackException

   // Assertion: topNode != null
   topNode = topNode.getNextNode();

   return top;
} // end pop
```

- Definition of `peek` and `pop`

Linked Implementation

- Definition of `isEmpty` and `clear`.

```
public boolean isEmpty()
{
   return topNode == null;
} // end isEmpty


public void clear()
{
   topNode = null;
} // end clear
```

Array-Based Stack Implementation

- Each operation involves top of stack
  - `push`
  - `pop`
  - `peek`
- End of the array easiest to access
  - Let this be top of stack
  - Let first entry be bottom of stack

Array-Based Stack Implementation

```
/** A class of stacks whose entries are stored in an array. */
public final class ArrayStack<T> implements StackInterface<T>
{
        private T[] stack;    // Array of stack entries
        private int topIndex; // Index of top entry
   private boolean integrityOK = false;
        private static final int DEFAULT_CAPACITY = 50;
        private static final int MAX_CAPACITY = 10000;

   public ArrayStack()
   {
      this(DEFAULT_CAPACITY);
   } // end default constructor

   public ArrayStack(int initialCapacity)
   {
      integrityOK = false;
      checkCapacity(initialCapacity);

      // The cast is safe because the new array contains null entries
      @SuppressWarnings("unchecked")
      T[] tempStack = (T[])new Object[initialCapacity];
      stack = tempStack;
         topIndex = -1;
      integrityOK = true;
  } // end constructor

//  < Implementations of the stack operations go here. >
//  . . .
} // end ArrayStack
```

Array-Based Stack Implementation

```
public void push(T newEntry)
{
   checkInyegrity();
   ensureCapacity();
   stack[topIndex + 1] = newEntry;
   topIndex++;
} // end push


private void ensureCapacity()
{
   if (topIndex >= stack.length - 1) // If array is full, double its size
   {
      int newLength = 2 * stack.length;
      checkCapacity(newLength);
      stack = Arrays.copyOf(stack, newLength);
   } // end if
} // end ensureCapacity
```

Array-Based Stack Implementation

```
public T peek()
{
   checkIntegrity();
   if (isEmpty())
      throw new EmptyStackException();
   else
      return stack[topIndex];
} // end peek

public T pop()
{
   checkIntegrity();
   if (isEmpty())
      throw new EmptyStackException();
   else
   {
      T top = stack[topIndex];
      stack[topIndex] = null;
      topIndex--;
      return top;
   } // end if
} // end pop
```

- Retrieving the top, operation is O(1)

Vector-Based Stack Implementation

- The class **`Vector`**
  - An object that behaves like a high-level array
    - Index begins with 0
    - Methods to access or set entries
    - Size will grow as needed
  - Has methods to add, remove, clear
    - Also methods to determine
      - Last element
      - Is the vector empty
      - Number of entries
- Use vector's methods to manipulate stack